

Chapter 7

Next.js

File-based routing · Server components · React with a server



SOURCE : freeCodeCamp x Scrimba – “Become a Fullstack Developer from Scratch”

MODULES : Build a Next.js app (43:06:47) · Rendering strategies and more (45:08:21)

COVERS : app router · routing · server vs client components · data fetching · optimisation

PROJECT : PrintForge – a 3D-model sharing site

⚡ THE KEY IDEA

Next.js gives React **a server**. Routes come from your folder structure instead of a router library, and components can **fetch data before they render** rather than after.

1 create-next-app

★★★★

Prompt	Answer & why
Use the App Router?	Yes – “a definite yes”, the most up-to-date version of Next.js
Put code in a <code>src</code> directory?	No – a preference, not a rule
Use Turbopack?	Yes locally; no in the course environment, where it doesn't run
Customise the import alias?	No – the default <code>@</code> is good enough

► What you get

File / folder	Purpose
<code>app/</code>	Absolutely necessary for a Next.js app. Holds the two required files, <code>layout.tsx</code> and <code>page.tsx</code>
<code>globals.css</code>	Truly global styles – font family, CSS variables, reset rules
<code>public/</code>	Optional. Anything inside is served <i>as-is as static files</i>
<code>next-env.d.ts</code>	A types definition file. <i>Never edit it</i> – it says so at the top
config files	<code>tsconfig</code> , Tailwind, PostCSS, <code>package.json</code> , <code>next.config</code>

⚠ DON'T LIVE IN `globals.css`

It is the right home for genuinely global rules. It is *not* your default way to style things – “there are better solutions than just cramming all of your CSS styles here”. This project uses *Tailwind; CSS modules* would also work.

RUNNING IT

`npm install`, then `npm run dev`. Because Next.js needs a back end running, the course uses an automated *runner* that issues both commands when a scrim loads – the same feedback you'd get from the terminal, without typing it each time. Expect the install to pull in *around 310 packages*.

⚡ THE TWO REQUIRED FILES

Everything else in a fresh project is replaceable. These two are not:

`app/layout.tsx` wraps every page – the shared shell, where global providers and site-wide chrome live.
`app/page.tsx` is the homepage. Delete either and the app *will not run*. The default `public/` SVGs, by contrast, are *placeholders the framework expects you to delete*.

2 Folders become URLs

★★★★★

THE RULE — TWO PARTS

1. Create a **folder** inside **app/** named after the route you want. On its own **this does not create a route**.
2. The folder needs a file called **page.tsx**, and that file must **default export** a component. The component's name doesn't matter.

```
// app/about/page.tsx → /about
export default function AboutPage() {
  return <h1>This is the about page</h1>;
}
```

- **app/page.tsx** — the **homepage**, because it isn't nested in any folder.
- Nest folders to nest routes: **app/about/mission/** → **/about/mission**.

⚡ THE PATTERN SCALES BY ITSELF

Once you've seen **/about**, nested routing needs no new syntax — you just **put a folder inside a folder**. Every site you've ever visited with a URL like **/about/mission** maps onto exactly this structure.

► Static assets

```
// 1 — reference the deployed path


// 2 — import it (preferred)
import heroImage from
  "@public/hero-image.png";

<img src={heroImage.src} />
```

Files in **public/** are served from the **root domain** — **public/hero.png** becomes **example.com/hero.png**.

Importing instead **teaches Next.js where the image lives** ahead of time. The import returns an object with **src**, **height**, **width** and blur data.

⚠ src ON AN IMPORTED ASSET

The Next.js **<Image>** component knows to read **.src** off that object for you. A native **** does **not** — you must write **heroImage.src** yourself.

WHY IMPORTING BEATS A STRING PATH

A string like **"/hero.png"** describes **where the file will eventually be deployed** — it is a promise about a future network request, and nothing checks it. An **import** is resolved at build time: get the path wrong and you find out **immediately**, not when a user hits a broken image.

⚠ A 404 IS THE SYMPTOM OF A MISSING page FILE

Navigate to a route you believe you created and get a **404**? Nine times out of ten the folder exists but the file inside is **misnamed** — **Page.tsx**, **index.tsx**, or a component that is exported but not **export default**. The folder gives the URL; the **default export inside page.tsx** gives the content.

3 Fetching before rendering

★★★★★

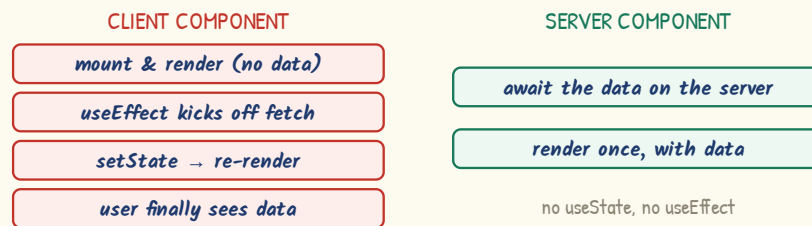


Fig 1 — two orders of operations for the same page

```
// app/posts/page.tsx — a server component
export default async function PostsPage() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  const data = await res.json();

  return <pre>{JSON.stringify(data, null, 2)}</pre>;
}
```

DEFINITION — SERVER COMPONENT

A component whose code is run **on the server**. Because it is rendered there, it can perform data fetching **before** it renders — so you **write asynchronous code as if it were synchronous** and simply **await** the data. In Next.js, components are server components **by default**.

⚡ WHAT IT SAVES YOU

The client version needs state, an effect, a re-render, plus loading and error handling — “an additional maybe seven to ten lines, if not more”. The server version is **async** + two **await** s.

⚠ API KEYS — READ THE CAVEAT

Because this fetch runs on the server before anything reaches the browser, a user **cannot inspect browser code and find your key**. You can also query a database directly with an ORM like Prisma from inside the component. But that is **not** blanket permission: commit the file to a public GitHub repo and you have a different problem entirely. **Use environment variables**.

"ONLY ON THE SERVER" — A USEFUL SIMPLIFICATION

The instructor says server component code runs **only on the server**, then flags that this is **not strictly true** — it just “helps get the point across”. Treat it as the working model now, and read Josh Comeau’s *Making Sense of Server Components* when you want the accurate picture. It is **a long read that may take a few passes**.

4 use client

★★★★★

```
"use client" // ← the very first line of the file

import { usePathname } from "next/navigation";

export default function Navbar() {
  const pathname = usePathname();
  return <NavLink href="/3d-models" isActive={pathname === "/3d-models"} />;
}
```

TWO WAYS A COMPONENT BECOMES A CLIENT COMPONENT

1. The file is marked with the `"use client"` directive at the **very top**. This comes from React, not Next.js.
2. It is **imported and rendered by another client component** – and that continues on down the chain. This is the **client boundary**.

⚠ THE INSTRUCTOR'S OWN CORRECTION

He first implies that marking the root layout `"use client"` turns the whole app into client components, then explicitly walks it back: *"that's actually not entirely true"*. Children **passed in** as the `children` prop are not converted – only components a client file **imports and renders by name**.

WHERE TO PUT THE BOUNDARY

Needing `usePathname` in a nav forces a choice: make the `NavLink` a client component and let it compare its own `href`, or make the `Navbar` one and pass an `isActive` boolean down. Both work. The instructor tested both and found **far less friction** making the `Navbar` the client component – more verbose, but "more clear and concise code" with a lot more control from the outside. **There isn't a right or wrong answer, but there certainly are trade-offs.**

⚠ WHERE DID MY console.log GO?

A log inside a **server** component prints to the **server's terminal**; inside a **client** component it prints to the **browser console**. Looking in the wrong one is a classic five-minute confusion.

⚡ WHAT FORCES A CLIENT COMPONENT

Anything that needs the **browser**: React hooks (`useState`, `useEffect`), Next.js client hooks like `usePathname`, and event handlers such as `onClick`. Everything else should stay on the server. **Push the boundary as far down the tree as you can** – mark the small interactive leaf, not the layout above it.

5 Images

★★★★

```
import Image from "next/image";
```

```
<Image src={heroImage} alt="..." width={742} height={742} />
```

⚠ width AND height MEAN SOMETHING ELSE HERE

On a native `<img>` they set the displayed size. On the Next.js `<Image>` they are the image's native pixel dimensions – they do not force a size. Next.js uses them to keep proportions correct while it resizes. Both are required, as is `alt`.

Optimisation	What it does
Resizing	Fits the container, keeps proportions, converts to an efficient format such as WebP where supported
Lazy loading	The image only starts loading once scrolled into the viewport – no network cost for images users never reach
No layout shift	Reserves space for the incoming image so text doesn't jank downward when it arrives

⚠ NOT AVAILABLE IN THE COURSE ENVIRONMENT

The `<Image>` component does not work in Scrimba's web-container system, so the lessons fall back to a native `<img>` with an imported asset. Use the real component when developing locally.

6 Fonts

★★★

- Import from `next/font/google`, register it as a CSS variable, then apply that variable – e.g. to every `h1` – `h6`.
- `next/font/local` does the same for a font file you ship yourself.

THE PATTERN BEHIND BOTH

Images and fonts are the same idea twice: tell the framework about the asset at build time and it can optimise what it knows about. Reference an asset by a runtime string and it can't. That is also why `import` ing an image beats writing its future URL – the framework gains information it can act on before the user ever arrives.

⚡ WHY BOTHER

Next.js does "a lot of amazing things behind the scenes" for fonts loaded its way – work you'd otherwise have to do by hand in any web app.

7 ⚡ QUICK REVISION

Thing	The one-line answer
Create a project	<code>create-next-app</code> – say yes to the App Router
<code>app/</code>	Required. Must contain <code>layout.tsx</code> and <code>page.tsx</code>
A new route	A folder named for the route, containing a <code>page.tsx</code>
<code>page.tsx</code>	Must default export a component. The name is irrelevant
Homepage	<code>app/page.tsx</code> – top level, not nested
Nested route	A folder inside a folder. No new syntax
<code>public/</code>	Served as-is from the root domain
Static asset	Prefer <code>import</code> ; read <code>.src</code> for a native <code></code>
Default component	A server component
Fetching data	Make the component <code>async</code> and <code>await</code> – no state, no effect
API keys	Safe from the browser, not safe in a public repo – use env vars
<code>"use client"</code>	Very first line of the file. Needed for hooks, state, events
Client boundary	Anything a client file imports and renders becomes client too
<code>usePathname</code>	From <code>next/navigation</code> . Client-side only
<code><Image></code>	<code>width</code> / <code>height</code> = native pixel size, not display size
Image wins	Resize · WebP · lazy load · no layout shift
Fonts	<code>next/font/google</code> or <code>next/font/local</code> , via a CSS variable

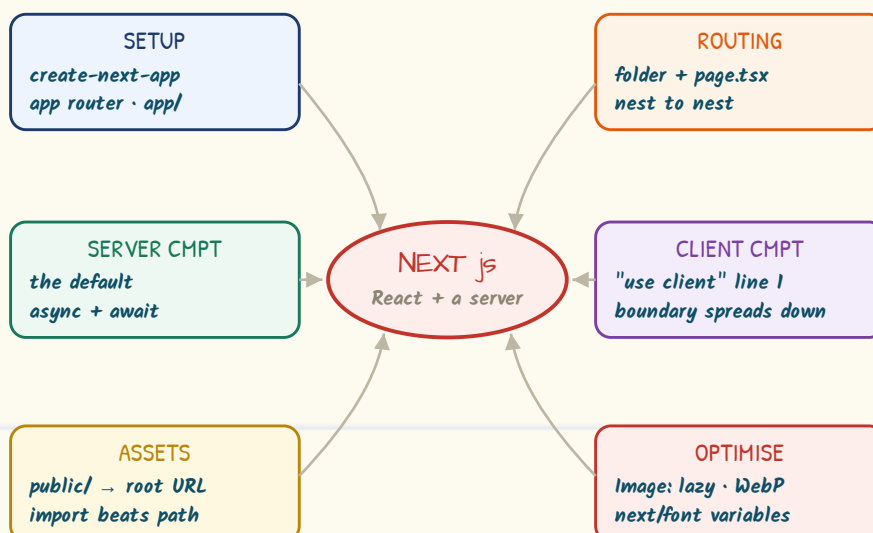
⚡ THE FOUR MOST FORGETTABLE DETAILS

1. A folder alone doesn't make a route – it needs `page.tsx`.
2. `"use client"` must be the **first line**, above the imports.
3. Children passed as a prop **don't** cross the client boundary; imported-and-rendered components do.
4. `console.log` lands in the terminal on the server and the browser on the client.

READING SOMEONE ELSE'S NEXT.JS APP

Start from the `app/` folder tree – it **is** the site map, so you can read every URL off the directory structure before opening a single file. Then grep for `"use client"`: those files, plus everything they import and render, are the **interactive parts**. Everything else runs on the server.

8 MIND MAP



⚡ THE THROUGH-LINE

Next.js keeps asking "can this happen on the server instead?" Routing moves from a library to the file system. Data fetching moves from an effect to an `await`. Image and font work moves from your build config into the framework. You opt back into the browser *deliberately*, one "use client" at a time.

Import	Gives you
<code>next/image</code>	The optimising <code><Image></code> component
<code>next/font/google</code>	A Google font, self-hosted and optimised
<code>next/font/local</code>	The same, for a font file you ship
<code>next/navigation</code>	<code>usePathname</code> – client-side only
<code>@/...</code>	The default import alias, rooted at the project
<code>react</code>	Hooks – but only inside a client component

That completes the seven chapters – HTML/CSS · JavaScript · React · Node · SQL · TypeScript · Next.js.